



**Universidad  
Europea** MADRID

**FACULTAD DE S.T.E.A.M**

---

**“Informe Proyecto Grupal Final”**

# **Proyecto Bases De Datos**

Para la asignatura de:

**Bases De Datos**

Presenta:

**Matias Bay**

**Xavi Carrera**

**Pablo Cabada**

**Jorge Amo**

**Pablo Cuenca**

# Memoria descriptiva del proyecto final

Proyecto: Mas Tienda, marketplace de segunda mano inspirado en Vinted.

Tecnologías principales: React/Vite, FastAPI, Supabase y PostgreSQL.

| Sección | Contenido                              |
|---------|----------------------------------------|
| 1       | Descripción del problema               |
| 2       | Diagrama E/R y tablas resultado        |
| 3       | Normalización                          |
| 4       | Construcción de la base de datos       |
| 5       | Seguridad                              |
| 6       | Consultas empotradas por pantalla      |
| 7       | Detalles de código de especial interés |
| 8       | División del trabajo                   |
| 9       | Uso de la IA                           |
| 10      | Conclusión                             |

## 1. Descripción del problema

El proyecto, llamado Mas Tienda, es una plataforma web de compraventa entre usuarios inspirada en el funcionamiento de Vinted. La aplicación permite registrarse, iniciar sesión, publicar artículos, consultar productos de otros usuarios, marcar favoritos, enviar mensajes, hacer ofertas, responder contraofertas y gestionar el perfil.

El problema que resuelve es centralizar en una misma aplicación las operaciones básicas de un marketplace de segunda mano: publicación de productos, descubrimiento de artículos, negociación, comunicación comprador-vendedor y administración de usuarios/productos.

Sin una base de datos bien diseñada, estas operaciones generarían duplicidades, estados incoherentes y problemas de seguridad. Por ejemplo, un artículo vendido podría seguir apareciendo como disponible, un usuario podría intentar modificar artículos ajenos o una conversación podría duplicarse varias veces para el mismo artículo o generar una base de datos gigantesca con datos mal gestionado de tal manera que los tiempos de consulta se dispararían.

La solución técnica se divide en un frontend en React/Vite y un backend en FastAPI conectado a Supabase que a su vez usa PostgreSQL. La base de datos es el centro del proyecto porque relaciona usuarios, artículos, fotos, favoritos, mensajes, ofertas, pedidos, incidencias y permisos. De tal manera que cada vez que se realiza una operación se usa la base de datos.

## 2. Diagrama E/R y tablas resultado

### 2.1 Aclaración sobre la agrupación por dominios

En este proyecto, dominio no significa un tipo DOMAIN de SQL ni una base de datos separada. La palabra se usa únicamente como una agrupación lógica por responsabilidad funcional para explicar mejor el modelo.

Los dominios son: usuarios y acceso; catálogo; interacción; compra, envío y soporte. Esta agrupación ayuda a leer el diagrama, pero el esquema final es único y coherente. Las claves foráneas cruzan dominios cuando el negocio lo necesita.

Esta aclaración corrige el punto señalado en la entrega anterior: los dominios no son entidades nuevas ni cambian la normalización; son una forma visual de organizar tablas relacionadas.

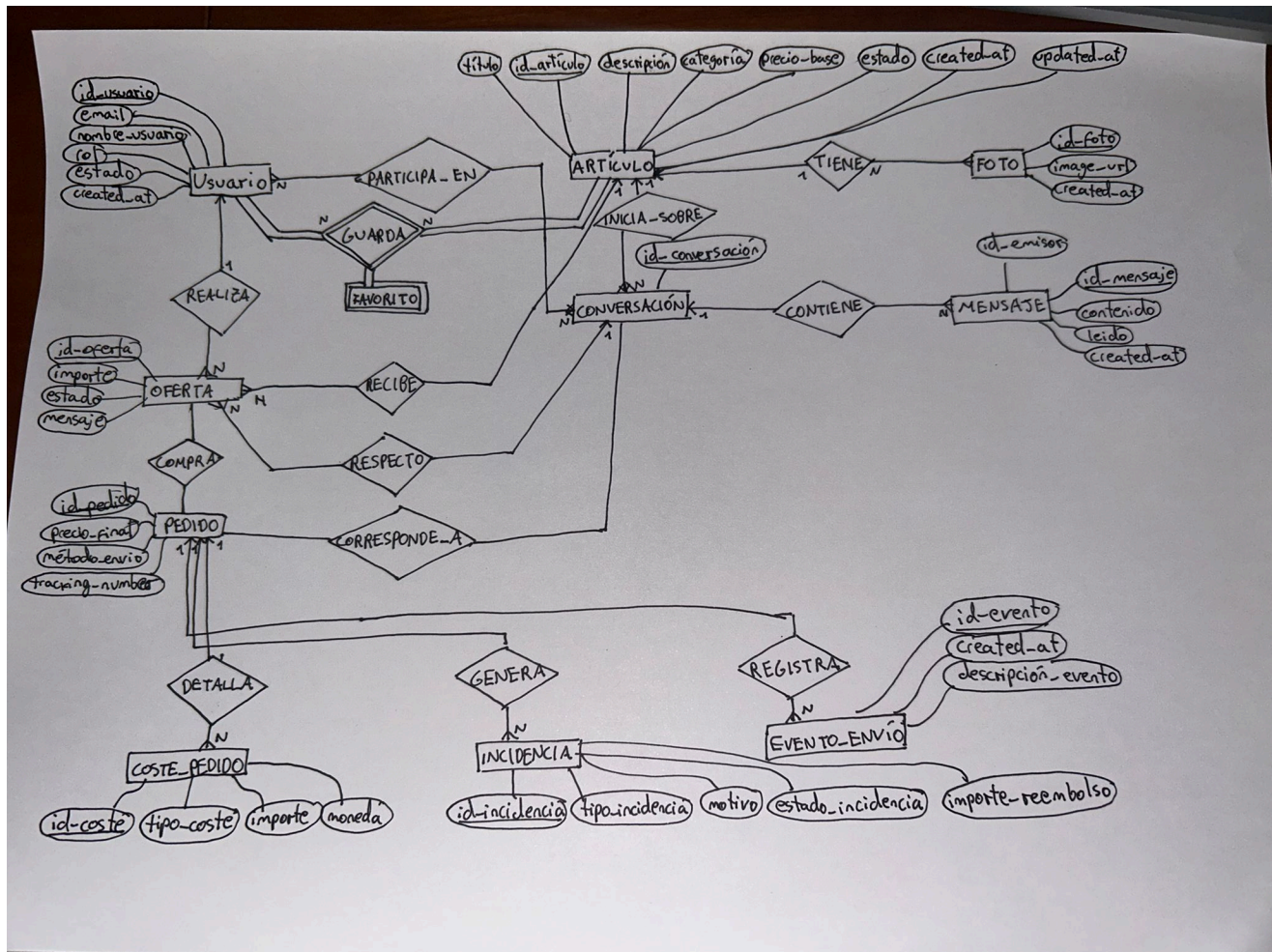


Figura 1. Diagrama E/R organizado por dominios lógicos.

## 2.2 Tablas resultado

| Tabla          | Función                                    | PK / relaciones principales                                                                           |
|----------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| usuarios       | Perfiles, email, estado y rol.             | PK id_usuario; se relaciona con artículos, ofertas, favoritos, conversaciones, mensajes y pedidos.    |
| articulos      | Productos publicados.                      | PK id_articulo; FK id_vendedor; se relaciona con fotos, ofertas, favoritos, conversaciones y pedidos. |
| fotos_articulo | Imágenes de artículos.                     | PK id_foto; FK id_articulo.                                                                           |
| ofertas        | Propuestas y contraofertas.                | PK id_oferta; FK id_comprador; FK id_articulo; FK ultimo_emisor_id.                                   |
| conversaciones | Chat entre dos usuarios sobre un artículo. | PK id_conversacion; FK id_usuario_1; FK id_usuario_2; FK id_articulo; UNIQUE usuarios-articulo.       |
| mensajes       | Mensajes del chat.                         | PK id_mensaje; FK id_conversacion; FK id_emisor.                                                      |
| favoritos      | Artículos guardados.                       | PK id_favorito; FK id_usuario; FK id_articulo; UNIQUE usuario-articulo.                               |
| pedidos        | Operaciones de compra.                     | PK id_pedido; FK id_comprador; FK id_articulo.                                                        |
| costes_pedidos | Desglose de costes del pedido.             | PK id_coste; FK id_pedido.                                                                            |
| eventos_envio  | Seguimiento del envío.                     | PK id_evento; FK id_pedido.                                                                           |

|             |                      |                                 |
|-------------|----------------------|---------------------------------|
| incidencias | Problemas postventa. | PK id incidencia; FK id pedido. |
|-------------|----------------------|---------------------------------|

```

erDiagram
    usuarios ||--o{ articulos : "publica"
    usuarios ||--o{ ofertas : "realiza"
    usuarios ||--o{ favoritos : "guarda"
    usuarios ||--o{ conversaciones : "participa"
    usuarios ||--o{ mensajes : "envia"
    usuarios ||--o{ pedidos : "compra"
    articulos ||--o{ fotos_articulo : "tiene"
    articulos ||--o{ ofertas : "recibe"
    articulos ||--o{ favoritos : "es_marcado"
    articulos ||--o{ conversaciones : "genera"
    articulos ||--o{ pedidos : "se_convierte"
    conversaciones ||--o{ mensajes : "contiene"
    pedidos ||--o{ costes_pedidos : "detalla"
    pedidos ||--o{ incidencias : "genera"
    pedidos ||--o{ eventos_envio : "registra"

```

## 3. Normalización

### 3.1 Esquema inicial tentativo

En una primera aproximación el sistema podía haberse diseñado con menos tablas y con información mezclada: usuarios, artículos con varias fotos en una columna, ofertas con datos repetidos, chats con mensajes incrustados y pedidos con costes e incidencias dentro de la misma tabla.

| Problema del esquema inicial                              | Corrección en el esquema final              |
|-----------------------------------------------------------|---------------------------------------------|
| Varias fotos dentro de un campo de artículos.             | Tabla fotos_articulo con una fila por foto. |
| Mensajes como lista dentro de chats.                      | Tablas conversaciones y mensajes.           |
| Costes mezclados dentro de pedidos.                       | Tabla costes_pedidos.                       |
| Eventos de envío dentro de pedidos.                       | Tabla eventos_envio.                        |
| Datos de usuario/artículo repetidos en ofertas y pedidos. | Uso de claves foráneas.                     |

### 3.2 Primera Forma Normal (1FN)

La 1FN exige valores atómicos. Por eso se separaron fotos, mensajes, costes, eventos e incidencias en tablas propias. Así se evita guardar listas o valores múltiples en un solo campo.

### 3.3 Segunda Forma Normal (2FN)

La 2FN exige que los atributos dependan completamente de la clave primaria. Como la mayoría de tablas tienen claves simples, la decisión principal fue no repetir atributos de otras entidades: ofertas guarda id\_artículo, mensajes guarda id\_emisor y favoritos solo relaciona usuario-artículo.

### 3.4 Tercera Forma Normal (3FN)

La 3FN exige eliminar dependencias transitivas. Por ejemplo, pedidos no guarda el nombre del vendedor; se obtiene desde articulos.id\_vendedor y usuarios. Conversaciones no guarda el título del artículo; se consulta desde articulos. El esquema final cumple 3fn porque cada tabla representa una entidad o relación concreta.

### 3.5 Esquema final tras normalización

- usuarios(id\_usuario PK, email UNIQUE, nombre\_usuario, estado, rol, created\_at)
- articulos(id\_articulo PK, id\_vendedor FK, titulo, descripcion, categoria, precio\_base, estado\_articulo, created\_at, updated\_at)

- fotos\_articulo(id\_foto PK, id\_articulo FK, image\_url, created\_at)
- ofertas(id\_oferta PK, id\_comprador FK, id\_articulo FK, ultimo\_emisor\_id FK, estado, importe, mensaje, created\_at, updated\_at)
- pedidos(id\_pedido PK, id\_comprador FK, id\_articulo FK, estado\_pedido, precio\_final, metodo\_envio, desde\_direccion, hacia\_direccion, tracking\_number, created\_at, updated\_at)
- costes\_pedidos(id\_coste PK, id\_pedido FK, tipo\_coste, importe, moneda, created\_at)
- incidencias(id\_incidencia PK, id\_pedido FK, tipo\_incidencia, motivo, estado\_incidencia, importe\_reembolso, requiere\_devolucion, created\_at, updated\_at)
- eventos\_envio(id\_evento PK, id\_pedido FK, descripcion\_evento, created\_at)
- conversaciones(id\_conversacion PK, id\_usuario\_1 FK, id\_usuario\_2 FK, id\_articulo FK, created\_at)
- mensajes(id\_mensaje PK, id\_conversacion FK, id\_emisor FK, contenido, leído, created\_at)
- favoritos(id\_favorito PK, id\_usuario FK, id\_articulo FK, created\_at)

## 4. Construcción de la base de datos

La base de datos está construida en PostgreSQL usando Supabase. Esta elección permite claves foráneas, enumerados, restricciones, vistas, índices y Row Level Security.

### 4.1 Tipos de datos y justificación

| Elemento       | Tipo usado               | Justificación                                                                                           |
|----------------|--------------------------|---------------------------------------------------------------------------------------------------------|
| IDs de usuario | UUID                     | Supabase Auth genera identificadores UUID; por eso usuarios.id_usuario se sincroniza con autenticación. |
| IDs internos   | BIGSERIAL                | Identificador autoincremental simple para artículos, fotos, ofertas, pedidos y eventos.                 |
| Email          | VARCHAR(255) UNIQUE      | Longitud suficiente y restricción contra duplicados.                                                    |
| Precios        | NUMERIC(10,2)            | Adecuado para importes monetarios y evita errores de coma flotante.                                     |
| Estados        | ENUM                     | Impide estados fuera del flujo definido.                                                                |
| Fechas         | TIMESTAMP WITH TIME ZONE | Trazabilidad temporal con zona horaria.                                                                 |
| Textos largos  | TEXT                     | Descripciones, mensajes e incidencias.                                                                  |
| Leído/no leído | BOOLEAN                  | Representación clara del estado de lectura.                                                             |

### 4.2 Restricciones implementadas

- PRIMARY KEY en todas las tablas.
- FOREIGN KEY entre usuarios, artículos, ofertas, conversaciones, mensajes, pedidos, costes, eventos e incidencias.
- UNIQUE(email) para evitar duplicar cuentas por correo.
- UNIQUE(id\_usuario, id\_articulo) en favoritos para evitar favoritos duplicados.
- UNIQUE(id\_usuario\_1, id\_usuario\_2, id\_articulo) en conversaciones para evitar chats duplicados.
- ON DELETE CASCADE en tablas dependientes como fotos, mensajes, favoritos, eventos, costes e incidencias.
- ON DELETE RESTRICT en pedidos para proteger transacciones ya creadas.

### 4.3 Vistas corregidas y coherentes con PostgreSQL

En la fase 3 se propusieron vistas para catálogo, transacciones e incidencias. En esta versión final se corrigen dos detalles: los estados se usan en minúscula porque así están definidos en los ENUM, y las vistas se escriben con sintaxis válida de PostgreSQL.

```

CREATE OR REPLACE VIEW vista_catalogo_publico AS
SELECT DISTINCT ON (a.id_articulo)
    a.id_articulo, a.titulo, a.descripcion, a.categoria, a.precio_base,
    f.image_url AS foto_principal,
    a.created_at AS fecha_publicacion
FROM articulos a
LEFT JOIN fotos_articulo f ON a.id_articulo = f.id_articulo
WHERE a.estado_articulo = 'disponible'
ORDER BY a.id_articulo, f.id_foto ASC;

CREATE OR REPLACE VIEW vista_resumen_transacciones AS
SELECT
    p.id_pedido, p.id_comprador, a.id_vendedor,
    a.titulo AS articulo_comprado,
    p.estado_pedido, p.metodo_envio, p.precio_final,
    COALESCE((SELECT SUM(cp.importe)
               FROM costes_pedidos cp
               WHERE cp.id_pedido = p.id_pedido), 0) AS costes_asociados,
    p.tracking_number,
    p.updated_at AS ultima_actualizacion
FROM pedidos p
JOIN articulos a ON p.id_articulo = a.id_articulo;

CREATE OR REPLACE VIEW vista_admin_incidencias AS
SELECT
    i.id_incidencia, i.tipo_incidencia, i.motivo,
    i.estado_incidencia, i.importe_reembolso,
    p.id_pedido, p.estado_pedido, a.id_vendedor, p.id_comprador
FROM incidencias i
JOIN pedidos p ON i.id_pedido = p.id_pedido
JOIN articulos a ON p.id_articulo = a.id_articulo;

```

## 4.4 Índices y justificación

| Índice                                 | Uso                                                             |
|----------------------------------------|-----------------------------------------------------------------|
| idx_articulos_estado_categoria_created | Optimiza catálogo por estado, categoría y fecha de publicación. |
| idx_articulos_vendedor                 | Acelera perfil del vendedor y gestión de artículos propios.     |
| idx_fotos_articulo                     | Acelera carga de fotos por artículo.                            |
| idx_ofertas_comprador_estado           | Acelera consulta de ofertas del comprador.                      |
| idx_ofertas_articulo_estado            | Acelera consulta de ofertas por artículo.                       |
| idx_pedidos_comprador_created          | Acelera historial de compras.                                   |
| idx_conversaciones_usuarios_articulo   | Acelera búsqueda de chats por usuarios y artículo.              |
| idx_mensajes_conversacion_lectura      | Acelera mensajes no leídos y carga de conversaciones.           |
| idx_eventos_pedido_fecha               | Acelera seguimiento cronológico de envíos.                      |

```

CREATE INDEX IF NOT EXISTS idx_articulos_estado_categoria_created
ON articulos(estado_articulo, categoria, created_at);
CREATE INDEX IF NOT EXISTS idx_articulos_vendedor ON articulos(id_vendedor);
CREATE INDEX IF NOT EXISTS idx_fotos_articulo ON fotos_articulo(id_articulo);
CREATE INDEX IF NOT EXISTS idx_ofertas_comprador_estado ON ofertas(id_comprador, estado,
updated_at);
CREATE INDEX IF NOT EXISTS idx_ofertas_articulo_estado ON ofertas(id_articulo, estado,
updated_at);
CREATE INDEX IF NOT EXISTS idx_pedidos_comprador_created ON pedidos(id_comprador, created_at);
CREATE INDEX IF NOT EXISTS idx_conversaciones_usuarios_articulo ON conversaciones(id_usuario_1,
id_usuario_2, id_articulo);
CREATE INDEX IF NOT EXISTS idx_mensajes_conversacion_lectura ON mensajes(id_conversacion,
leido, id_emisor);
CREATE INDEX IF NOT EXISTS idx_eventos_pedido_fecha ON eventos envio(id_pedido, created_at);

```

## 5. Seguridad

La seguridad se trabaja en tres capas: autenticación, autorización en backend y reglas en base de datos.



El backend usa OAuth2PasswordBearer y Supabase Auth. Cuando el usuario inicia sesión, Supabase devuelve un token. En el frontend, Axios añade ese token a cada petición mediante Authorization: Bearer <token>.

Los administradores se controlan con el campo rol de usuarios. La función `get_admin_user` comprueba que el usuario autenticado tenga rol = admin. En SQL, `es_admin()` permite aplicar este control en políticas RLS.

```
CREATE OR REPLACE FUNCTION es_admin()
RETURNS BOOLEAN AS $$
BEGIN
    RETURN EXISTS (
        SELECT 1 FROM usuarios
        WHERE id_usuario = auth.uid()
        AND rol = 'admin'
    );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

ALTER TABLE articulos ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Cualquiera puede ver artículos disponibles"
ON articulos FOR SELECT
USING (estado_articulo = 'disponible' OR auth.uid() = id_vendedor OR es_admin());

CREATE POLICY "Usuarios pueden crear artículos"
ON articulos FOR INSERT
WITH CHECK (auth.uid() = id_vendedor);

CREATE POLICY "Vendedores pueden editar sus artículos"
ON articulos FOR UPDATE
USING (auth.uid() = id_vendedor OR es_admin());
```

## 6. Consultas empotradas por pantalla

La aplicación no ejecuta SQL escrito directamente en el frontend. Usa Axios para llamar al backend y el backend usa el cliente de Supabase. A continuación se muestran las consultas SQL equivalentes a cada pantalla.

### 6.1 Registro e inicio de sesión

Pantalla: AuthPage.jsx. Rutas: `/api/auth/register` y `/api/auth/login`.

```
INSERT INTO usuarios(id_usuario, email, nombre_usuario, estado, rol)
VALUES (:id_usuario, :email, :nombre_usuario, 'activo', 'user')
ON CONFLICT (id_usuario) DO UPDATE
SET email = EXCLUDED.email,
    nombre_usuario = EXCLUDED.nombre_usuario;

SELECT * FROM usuarios WHERE id_usuario = :id_usuario;
```

### 6.2 Página principal / catálogo

Pantalla: HomePage.jsx. Ruta: `GET /api/items`.

```
SELECT a.*, f.image_url, u.nombre_usuario, u.email
FROM articulos a
LEFT JOIN fotos_articulo f ON a.id_articulo = f.id_articulo
JOIN usuarios u ON a.id_vendedor = u.id_usuario
WHERE a.estado_articulo = 'disponible'
AND (:categoria IS NULL OR a.categoria = :categoria)
```

```
AND (:search IS NULL OR a.titulo ILIKE '%' || :search || '%'
    OR a.descripcion ILIKE '%' || :search || '%')
ORDER BY a.created_at DESC
LIMIT :limit;
```

### 6.3 Detalle del artículo

Pantalla: ArticleDetailPage.jsx. Ruta: GET /api/items/{id}.

```
SELECT a.*, f.*, u.nombre_usuario, u.email
FROM articulos a
LEFT JOIN fotos_articulo f ON a.id_articulo = f.id_articulo
JOIN usuarios u ON a.id_vendedor = u.id_usuario
WHERE a.id_articulo = :id_articulo;
```

### 6.4 Publicar producto y subir imágenes

Pantalla: PublishPage.jsx. Rutas: POST /api/items y POST /api/items/{id}/images.

```
INSERT INTO articulos(id_vendedor, titulo, descripcion, categoria, precio_base,
estado_articulo)
VALUES (:id_vendedor, :titulo, :descripcion, :categoria, :precio_base, 'disponible')
RETURNING *;

INSERT INTO fotos_articulo(id_articulo, image_url)
VALUES (:id_articulo, :image_url);
```

### 6.5 Favoritos

Pantallas: HomePage.jsx, ArticleDetailPage.jsx y ProfilePage.jsx. Rutas: /api/favorites.

```
SELECT f.*, a.*
FROM favoritos f
JOIN articulos a ON f.id_articulo = a.id_articulo
WHERE f.id_usuario = :id_usuario;

INSERT INTO favoritos(id_usuario, id_articulo)
VALUES (:id_usuario, :id_articulo)
ON CONFLICT (id_usuario, id_articulo) DO NOTHING;

DELETE FROM favoritos
WHERE id_usuario = :id_usuario AND id_articulo = :id_articulo;
```

### 6.6 Mensajes y conversaciones

Pantalla: MessagesPage.jsx. Rutas: /api/messages y /api/messages/conversations.

```
SELECT id_conversacion
FROM conversaciones
WHERE id_usuario_1 = :usuario_menor
    AND id_usuario_2 = :usuario_mayor
    AND id_articulo = :id_articulo;

INSERT INTO conversaciones(id_usuario_1, id_usuario_2, id_articulo)
VALUES (:usuario_menor, :usuario_mayor, :id_articulo)
RETURNING id_conversacion;

INSERT INTO mensajes(id_conversacion, id_emisor, contenido, leído)
VALUES (:id_conversacion, :id_emisor, :contenido, false);
```



## 6.7 Ofertas y contraofertas

Pantallas: ArticleDetailPage.jsx, MessagesPage.jsx y CounterOfferPage.jsx. Rutas: /api/offers.

```
INSERT INTO ofertas(id_comprador, id_articulo, ultimo_emisor_id, estado, importe, mensaje)
VALUES (:id_comprador, :id_articulo, :id_comprador, 'pendiente', :importe, :mensaje)
RETURNING *;
```

```
UPDATE ofertas
SET importe = :nuevo_importe,
    mensaje = :mensaje,
    ultimo_emisor_id = :id_usuario,
    estado = 'pendiente',
    updated_at = CURRENT_TIMESTAMP
WHERE id_oferta = :id_oferta;
```

```
UPDATE ofertas SET estado = 'aceptada', updated_at = CURRENT_TIMESTAMP
WHERE id_oferta = :id_oferta;
```

```
UPDATE articulos SET estado_articulo = 'vendido', updated_at = CURRENT_TIMESTAMP
WHERE id_articulo = :id_articulo;
```

## 6.8 Perfil del usuario

Pantalla: ProfilePage.jsx. Rutas: /api/users/me, /api/users/me/items, /api/users/me/favorites y /api/users/me/purchases.

```
SELECT * FROM usuarios WHERE id_usuario = :id_usuario;
```

```
SELECT a.*, f.*
FROM articulos a
LEFT JOIN fotos_articulo f ON a.id_articulo = f.id_articulo
WHERE a.id_vendedor = :id_usuario;
```

```
SELECT p.*, a.*
FROM pedidos p
JOIN articulos a ON p.id_articulo = a.id_articulo
WHERE p.id_comprador = :id_usuario;
```

## 6.9 Administración

Pantalla: AdminPage.jsx. Rutas: /api/admin/items, /api/admin/users y /api/admin/offers.

```
SELECT a.*, u.nombre_usuario, u.email
FROM articulos a
JOIN usuarios u ON a.id_vendedor = u.id_usuario
ORDER BY a.created_at DESC;
```

```
UPDATE usuarios SET estado = 'suspendido' WHERE id_usuario = :id_usuario;
UPDATE usuarios SET estado = 'activo' WHERE id_usuario = :id_usuario;
DELETE FROM articulos WHERE id_articulo = :id_articulo;
UPDATE articulos SET categoria = :categoria WHERE id_articulo = :id_articulo;
```

## 7. Detalles de código de especial interés

El proyecto separa frontend y backend. vinted-frontend contiene React/Vite, páginas, componentes, contexto de autenticación y cliente Axios. vinted-backend contiene FastAPI, routers, modelos Pydantic, seguridad y conexión a Supabase.

El cliente Axios añade automáticamente el token JWT al encabezado de cada petición. Así no hay que repetir la autenticación en cada pantalla.

El backend usa dos clientes de Supabase: `supabase_client` para operaciones normales y `supabase_admin` para operaciones internas validadas por el servidor.

Cuando se crea una oferta, el backend crea o recupera la conversación entre comprador y vendedor y añade un mensaje automático con el importe ofertado.

La gestión de imágenes se hace con Supabase Storage: la imagen se guarda fuera de la base de datos y en `fotos_articulo` solo se guarda la URL pública.

El fichero entregable `nomgrupo.py` puede ser `correccaminos.py`, basado en `run_dev.py`, para arrancar backend y frontend.

## 8. División del trabajo y gestión del equipo

El equipo se organizó por responsabilidades, aunque todos revisaron el funcionamiento general para poder superar el examen de autoría.

| Integrante                 | Responsabilidad principal                                                                     |
|----------------------------|-----------------------------------------------------------------------------------------------|
| Jorge Amo Barrio           | Coordinación, backend, despliegue final y estructura general.                                 |
| Pablo Cabada Pérez         | Backend, APIs, validaciones, revisión de arquitectura y funciones de administración.          |
| Xavi Carrera Guezala Gochi | Frontend, diseño de interfaz, experiencia de usuario y preparación visual de la presentación. |
| Matías Bay Rodríguez       | Base de datos, testing, pruebas de integración y rendimiento de consultas.                    |
| Pablo Cuenca               | Testing y ayuda a la documentación general.                                                   |

La gestión se realizó mediante reuniones de seguimiento. Primero se definió la arquitectura y el reparto; después se trabajó en autenticación, conexión frontend-backend, productos, pruebas, corrección de errores y cierre final. En la última fase se añadió el rol de administrador porque faltaba cumplir completamente el requisito de distintos tipos de usuario.

## 9. Uso de la IA

La IA se utilizó como herramienta de apoyo, no como sustituto del trabajo del equipo. Se usó para generar prompts iniciales, proponer organización de React/Vite y FastAPI, crear el cliente Axios con token JWT, sugerir pantallas y revisar ideas de diseño visual. El equipo revisó, adaptó y conectó el código a su propio proyecto.

- Setup y conexión frontend-backend.
- Autenticación global y páginas de login/registro.
- Layout y navegación.
- Página principal del catálogo.
- Detalle de producto.
- Subida de producto e imágenes.

- Pulido visual.
- Generación de supabase
- Solventación de errores de código.

## 10. Conclusion

Este proyecto ha permitido aplicar los contenidos principales de Bases de Datos en un caso realista: modelo E/R, transformación a tablas, claves primarias y foráneas, normalización, restricciones, consultas, vistas, índices y seguridad.

La parte más útil fue comprobar que la base de datos no es algo aislado. Cada pantalla necesita una consulta concreta y cada consulta depende de que el modelo esté bien diseñado.

El resultado final es una aplicación tipo marketplace con frontend, backend y base de datos conectados. La entrega final queda reforzada porque ahora hay coherencia entre diagrama E/R, normalización, SQL, consultas y código.